

Banco de ejercicios adicionales (soluciones al final)

Todos independientes de los anteriores. Niveles: Básico → Intermedio → Avanzado. Al final tienes soluciones compactas.

Básico

1. **De listas a dict:** dadas `claves = ["es", "fr", "it"]` y `valores = ["España", "Francia", "Italia"]`, crea `pais_por_codigo`.
2. **Filtrar claves:** dado `d = {"a": 1, "b": None, "c": 3, "d": None}`, elimina las claves con valor `None`.
3. **Valores por defecto:** escribe `get_config(d, k, default)` que devuelva `d[k]` o `default` sin lanzar error ni mutar el dict.
4. **Contar longitudes:** a partir de `palabras = ["python", "dict", "etl", "pipeline"]`, construye `{palabra: len(palabra)}`.
5. **Actualizar in-place:** con `a = {"x": 1}` y `b = {"x": 9, "y": 2}`, actualiza `a` con prioridad de `b`.

Intermedio

6. **Agrupar por inicial:** dado `nombres`, genera `grupo = {inicial: [nombres...]}` (usa `setdefault` o `defaultdict`).
7. **Invertir con colisiones:** convierte `d = {"a": 1, "b": 2, "c": 1}` en `{1: ["a", "c"], 2: ["b"]}`.
8. **Normalizar claves:** de `raw = {" Nombre ": "Ana", "PAÍS": "ES"}` produce `{"nombre": "Ana", "pais": "ES"}` usando `strip()` y `casefold()`.
9. **Join por id:** une `usuarios = [{"id":1, "n":"Ana"}, {"id":2, "n":"Luis"}]` con `scores = [{"id":2, "s":88}, {"id":1, "s":95}]` → `{1:{...}, 2:{...}}` indexado por `id`.
10. **Aplanar nivel 1:** `{"user": {"id": 7, "name": "Ana"}, "meta": {"lang": "es"}}` → `{"user.id":7, "user.name": "Ana", "meta.lang":"es"}`.

Avanzado

11. **Índice invertido:** dado `docs = {1: "ai etl ai", 2: "etl kafka"}`, construye `{token: set(doc_ids)}`.
12. **Co-ocurrencias:** con `baskets = [{"leche", "pan"}, {"pan", "huevos"}, {"leche", "huevos"}]` crea un dict de dicts con conteos de pares.
13. **Deduplicar por clave con regla:** de `items = [{"id":1, "score":0.7}, {"id":1, "score":0.9}, {"id":2, "score":0.6}]` quédate con el mayor `score` por `id`.
14. **Shallow vs deep:** demuestra (con código) cómo `copy()` y `deepcopy()` se comportan en `{"perfil":{"skills":["py"]}}`.
15. **Validación TypedDict:** define un `TypedDict Producto` (`id:int, nombre:str, precio:float`) y una función `valida_producto(p: dict) -> bool` que verifique tipos básicos.

